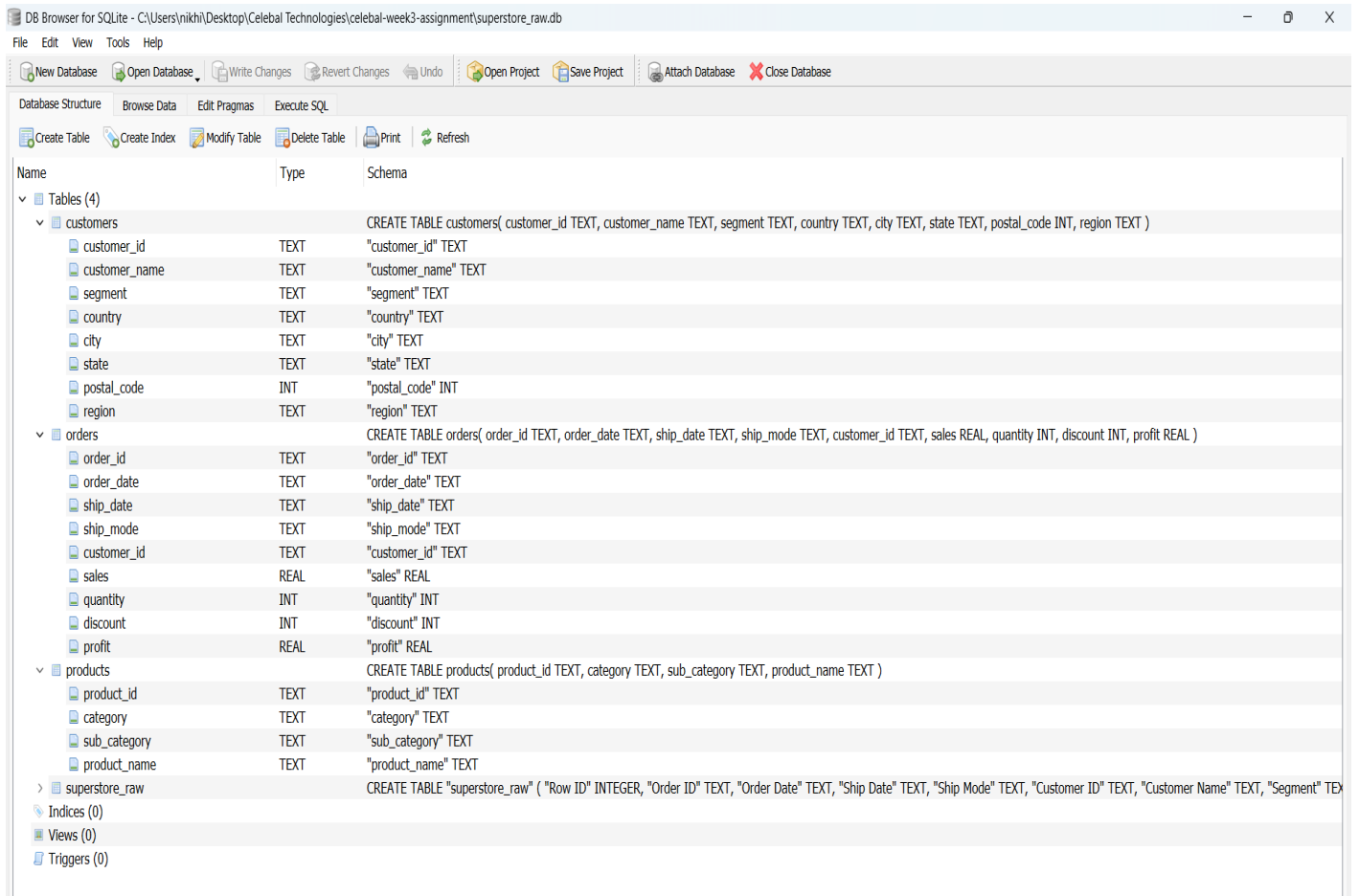


Loaded Superstore dataset into a table (superstore_raw).

-Create tables (customers, orders, products) from the dataset. Insert data into these tables using SELECT DISTINCT.



The screenshot shows the DB Browser for SQLite interface. The title bar indicates the database file is located at C:\Users\nikhil\Desktop\Celebal Technologies\celebal-week3-assignment\superstore_raw.db. The menu bar includes File, Edit, View, Tools, and Help. The toolbar contains icons for New Database, Open Database, Write Changes, Revert Changes, Undo, Open Project, Save Project, Attach Database, and Close Database. The main window is divided into three tabs: Database Structure, Browse Data, and Edit Pragma, with the Database Structure tab selected. Below the tabs, there are icons for Create Table, Create Index, Modify Table, Delete Table, Print, and Refresh. The Database Structure pane shows a tree view of the database contents. Under 'Tables (4)', there are four tables: customers, orders, products, and superstore_raw. Each table entry shows its name, type, and the SQL CREATE statement. The 'superstore_raw' table is expanded, showing its columns: Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, and Sales.

Name	Type	Schema
Tables (4)		
customers		CREATE TABLE customers(customer_id TEXT, customer_name TEXT, segment TEXT, country TEXT, city TEXT, state TEXT, postal_code INT, region TEXT)
customer_id	TEXT	"customer_id" TEXT
customer_name	TEXT	"customer_name" TEXT
segment	TEXT	"segment" TEXT
country	TEXT	"country" TEXT
city	TEXT	"city" TEXT
state	TEXT	"state" TEXT
postal_code	INT	"postal_code" INT
region	TEXT	"region" TEXT
orders		CREATE TABLE orders(order_id TEXT, order_date TEXT, ship_date TEXT, ship_mode TEXT, customer_id TEXT, sales REAL, quantity INT, discount INT, profit REAL)
order_id	TEXT	"order_id" TEXT
order_date	TEXT	"order_date" TEXT
ship_date	TEXT	"ship_date" TEXT
ship_mode	TEXT	"ship_mode" TEXT
customer_id	TEXT	"customer_id" TEXT
sales	REAL	"sales" REAL
quantity	INT	"quantity" INT
discount	INT	"discount" INT
profit	REAL	"profit" REAL
products		CREATE TABLE products(product_id TEXT, category TEXT, sub_category TEXT, product_name TEXT)
product_id	TEXT	"product_id" TEXT
category	TEXT	"category" TEXT
sub_category	TEXT	"sub_category" TEXT
product_name	TEXT	"product_name" TEXT
superstore_raw		CREATE TABLE "superstore_raw" ("Row ID" INTEGER, "Order ID" TEXT, "Order Date" TEXT, "Ship Date" TEXT, "Ship Mode" TEXT, "Customer ID" TEXT, "Customer Name" TEXT, "Segment" TEXT, "Sales" REAL)
Indices (0)		
Views (0)		
Triggers (0)		

Checked Weather The Table Created Or Not

DB Browser for SQLite - C:\Users\nikhi\Desktop\Celebal Technologies\celebal-week3-assignment\superstore_ra...

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Attach Database

Database Structure Browse Data Edit Pragmas Execute SQL

SQL 1*

```

1 SELECT name
2 FROM sqlite_master
3 WHERE type='table';

```

	name
1	superstore_raw
2	customers
3	orders
4	products

UTF-8

Subquery – Orders Above Average Sales

DB Browser for SQLite - C:\Users\nikhi\Desktop\Celebal Technologies\celebal-week3-assignment\superstore_raw.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

SQL 1*

```

1 SELECT *
2 FROM orders
3 WHERE sales >
4 (
5     SELECT AVG(sales)
6     FROM orders
7 );

```

	order_id	order_date	ship_date	ship_mode	customer_id	sales	quantity	discount	profit
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	261.96	2	0	41.9136
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	731.94	3	0	219.582
3	US-2015-108966	10/11/2015	10/18/2015	Standard Class	EG-20335	957.5775	5	0.45	-383.031
4	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	907.192	6	0.2	90.7152
5	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	1706.194	9	0.2	85.3092
6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	911.424	4	0.2	68.3568
7	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	407.976	3	0.2	132.5922
8	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19078	668.88	6	0	13.3176
9	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	1044.43	3	0	240.2649
10	US-2015-150430	9/17/2015	9/21/2015	Standard Class	TB-21520	3083.43	7	0.5	-1665.0522
11	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	1097.544	7	0.2	123.4737

Subquery – Highest Order Per Customer

DB Browser for SQLite - C:\Users\nikh\Desktop\Celebal Technologies\celebal-week3-assignment\superstore_raw.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

```
1 SELECT *
2 FROM orders o
3 WHERE sales =
4 (
5     SELECT MAX(sales)
6     FROM orders
7     WHERE customer_id = o.customer_id
8 );
```

	order_id	order_date	ship_date	ship_mode	customer_id	sales	quantity	discount	profit
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	731.94	3	0	219.592
2	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	957.5775	5	0.45	-383.031
3	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	1706.194	9	0.2	85.3092
4	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	1044.63	3	0	240.2649
5	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	3083.43	7	0.5	-1665.0522
6	CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	1097.544	7	0.2	123.4737
7	CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	532.3992	3	0.32	-46.9764
8	CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	1029.95	5	0	298.6855
9	CA-2016-111682	6/17/2016	6/18/2016	First Class	TB-21055	319.41	5	0.1	7.098
10	CA-2014-106376	12/5/2014	12/10/2014	Standard Class	BS-11590	1113.024	8	0.2	111.3024
11	CA-2015-129476	10/15/2015	10/20/2015	Standard Class	PA-19060	339.96	5	0.2	67.992

UTF

CTE – Total Sales Per Customer

DB Browser for SQLite - C:\Users\nikh\Desktop\Celebal Technologies\celebal-week3-assignment\superstore_ra...

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Attach Database

Database Structure Browse Data Edit Pragma Execute SQL

```
1 WITH customer_sales AS
2 (
3     SELECT
4         customer_id,
5         SUM(sales) AS total_sales
6     FROM orders
7     GROUP BY customer_id
8 )
9 SELECT *
10 FROM customer_sales
11 ORDER BY total_sales DESC;
```

	customer_id	total_sales
1	SM-20320	25043.05
2	TC-20980	19052.218
3	RB-19360	15117.339
4	TA-21385	14595.62
5	AB-10105	14473.571
6	KL-16645	14175.229
7	SC-20095	14142.334
8	HL-15040	12873.298
9	SE-20110	12209.438
10	CC-12370	12129.072
11	TS-21370	11891.751

UTF-8

Window Function – ROW_NUMBER()

The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following query:

```
1 SELECT
2     customer_id,
3     sales,
4     ROW_NUMBER() OVER
5     (
6         PARTITION BY customer_id
7         ORDER BY sales DESC
8     ) AS row_num
9 FROM orders;
```

The results pane displays the following data:

	customer_id	sales	row_num
1	AA-10315	3930.072	1
2	AA-10315	673.568	2
3	AA-10315	431.976	3
4	AA-10315	362.94	4
5	AA-10315	52.98	5
6	AA-10315	41.72	6
7	AA-10315	26.96	7
8	AA-10315	14.94	8
9	AA-10315	14.56	9
10	AA-10315	11.54	10
11	AA-10315	2.304	11

Window Function – RANK()

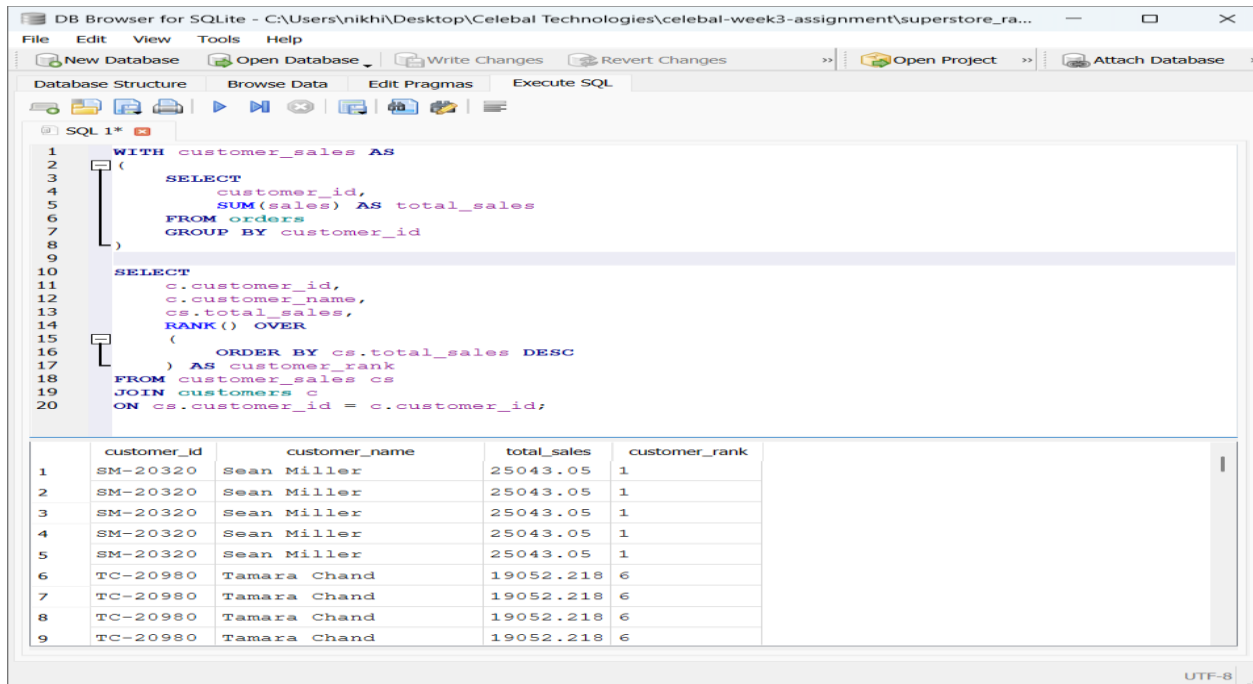
The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following query:

```
1 SELECT
2     customer_id,
3     SUM(sales) AS total_sales,
4     RANK() OVER
5     (
6         ORDER BY SUM(sales) DESC
7     ) AS sales_rank
8 FROM orders
9 GROUP BY customer_id;
```

The results pane displays the following data:

	customer_id	total_sales	sales_rank
1	SM-20320	25043.05	1
2	TC-20980	19052.218	2
3	RB-19360	15117.339	3
4	TA-21385	14595.62	4
5	AB-10105	14473.571	5
6	KL-16645	14175.229	6
7	SC-20095	14142.334	7
8	HL-15040	12873.298	8
9	SE-20110	12209.438	9
10	CC-12370	12129.072	10
11	TS-21370	11891.751	11
12	GT-14710	11820.12	12
13	BM-11140	11789.63	13

JOIN + CTE + Window Function



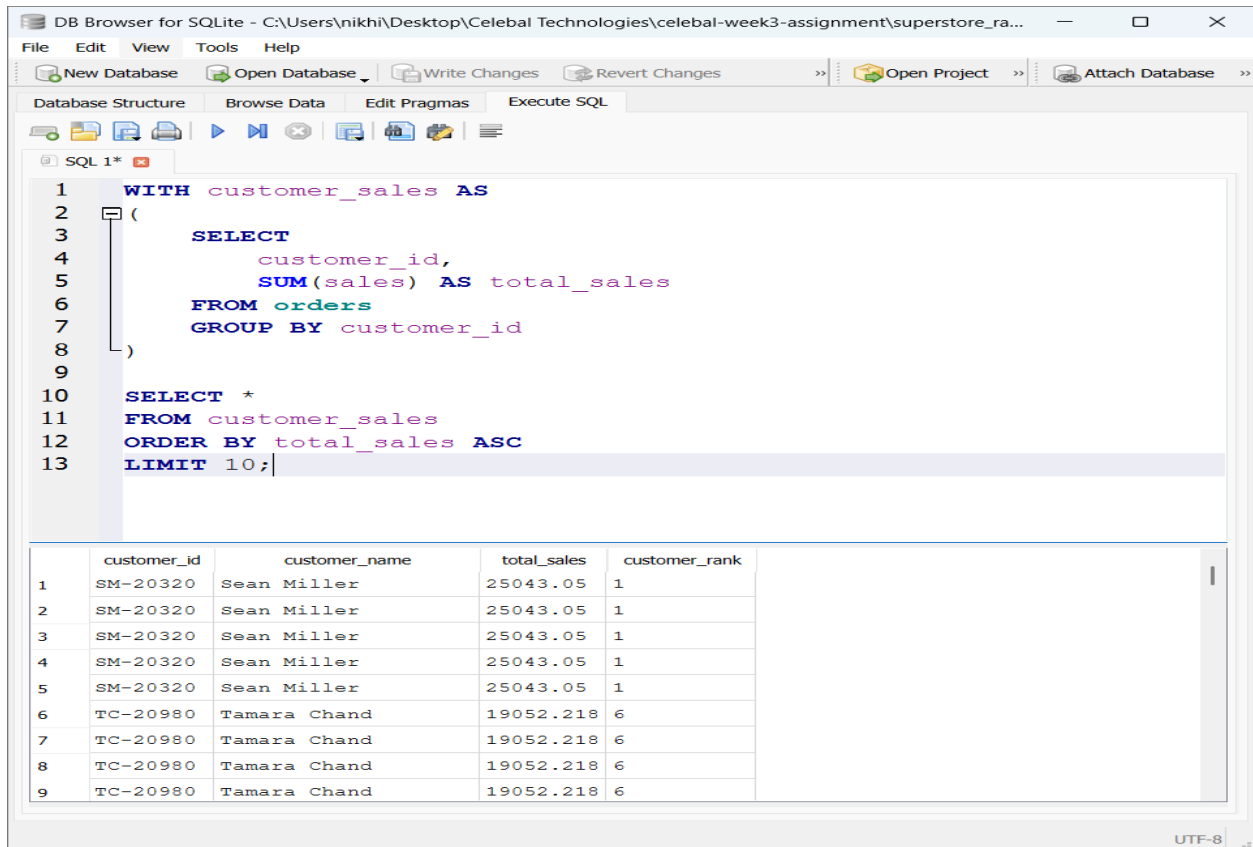
The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following query:

```
1 WITH customer_sales AS
2 (
3     SELECT
4         customer_id,
5         SUM(sales) AS total_sales
6     FROM orders
7     GROUP BY customer_id
8 )
9
10 SELECT
11     c.customer_id,
12     c.customer_name,
13     cs.total_sales,
14     RANK() OVER
15     (
16         ORDER BY cs.total_sales DESC
17     ) AS customer_rank
18 FROM customer_sales cs
19 JOIN customers c
20 ON cs.customer_id = c.customer_id;
```

The result set displays the top 10 customers based on total sales, ranked in descending order:

	customer_id	customer_name	total_sales	customer_rank
1	SM-20320	Sean Miller	25043.05	1
2	SM-20320	Sean Miller	25043.05	1
3	SM-20320	Sean Miller	25043.05	1
4	SM-20320	Sean Miller	25043.05	1
5	SM-20320	Sean Miller	25043.05	1
6	TC-20980	Tamara Chand	19052.218	6
7	TC-20980	Tamara Chand	19052.218	6
8	TC-20980	Tamara Chand	19052.218	6
9	TC-20980	Tamara Chand	19052.218	6

Top 10 Customers



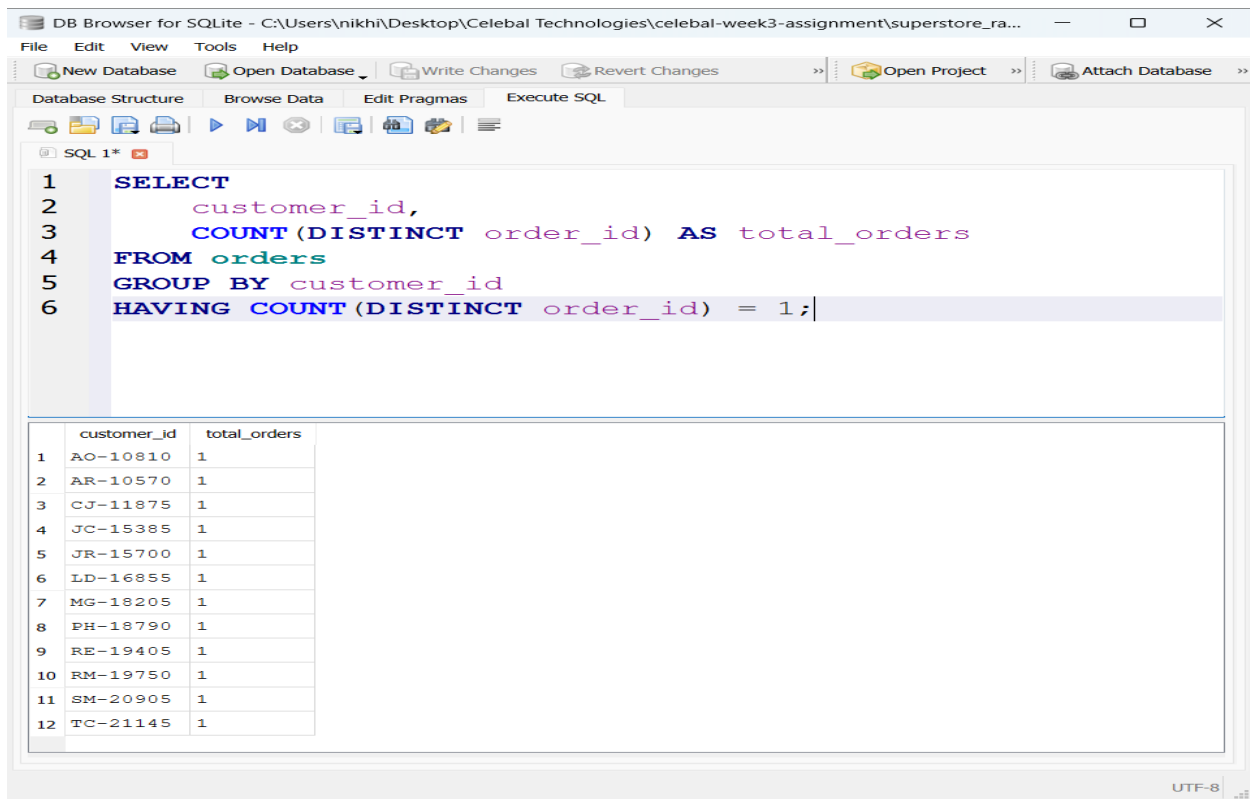
The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following query:

```
1 WITH customer_sales AS
2 (
3     SELECT
4         customer_id,
5         SUM(sales) AS total_sales
6     FROM orders
7     GROUP BY customer_id
8 )
9
10 SELECT *
11 FROM customer_sales
12 ORDER BY total_sales ASC
13 LIMIT 10;
```

The result set displays the top 10 customers based on total sales, ranked in ascending order:

	customer_id	customer_name	total_sales	customer_rank
1	SM-20320	Sean Miller	25043.05	1
2	SM-20320	Sean Miller	25043.05	1
3	SM-20320	Sean Miller	25043.05	1
4	SM-20320	Sean Miller	25043.05	1
5	SM-20320	Sean Miller	25043.05	1
6	TC-20980	Tamara Chand	19052.218	6
7	TC-20980	Tamara Chand	19052.218	6
8	TC-20980	Tamara Chand	19052.218	6
9	TC-20980	Tamara Chand	19052.218	6

Single-Order Customers



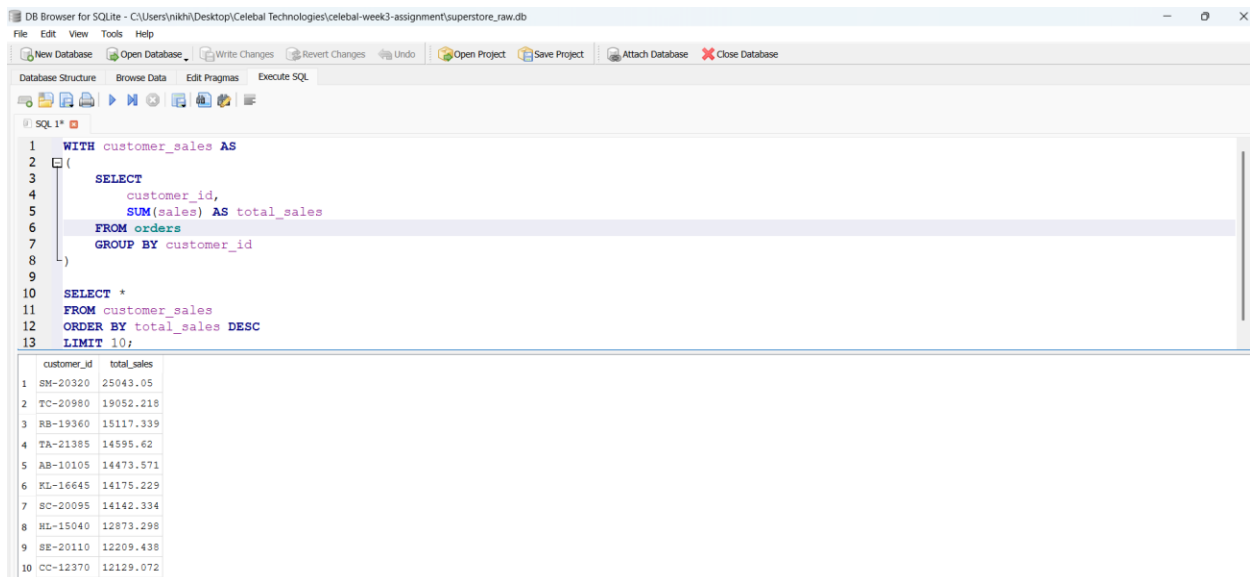
The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following query:

```
1 SELECT
2     customer_id,
3     COUNT(DISTINCT order_id) AS total_orders
4 FROM orders
5 GROUP BY customer_id
6 HAVING COUNT(DISTINCT order_id) = 1;
```

The results table displays 12 customers, each with a unique customer_id and a total_orders count of 1.

	customer_id	total_orders
1	AO-10810	1
2	AR-10570	1
3	CJ-11875	1
4	JC-15385	1
5	JR-15700	1
6	LD-16855	1
7	MG-18205	1
8	PH-18790	1
9	RE-19405	1
10	RM-19750	1
11	SM-20905	1
12	TC-21145	1

Lowest 10 Customers



The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following query:

```
1 WITH customer_sales AS
2 (
3     SELECT
4         customer_id,
5         SUM(sales) AS total_sales
6     FROM orders
7     GROUP BY customer_id
8 )
9
10 SELECT *
11 FROM customer_sales
12 ORDER BY total_sales DESC
13 LIMIT 10;
```

The results table displays the top 10 customers with the highest total sales, ordered from highest to lowest.

	customer_id	total_sales
1	SM-20320	25043.05
2	TC-20980	19052.218
3	RB-19360	15117.339
4	TA-21385	14595.62
5	AB-10105	14473.571
6	KL-16645	14175.229
7	SC-20095	14142.334
8	HL-15040	12873.298
9	BE-20110	12209.438
10	CC-12370	12129.072

Customers with Above-Average Total Sales

DB Browser for SQLite - C:\Users\nikhi\Desktop\Celebal Technologies\celebal-week3-assignment\superstore_ra...

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Attach Database

Database Structure Browse Data Edit Pragmas Execute SQL

SQL 1*

```
1 WITH customer_sales AS
2 (
3     SELECT
4         customer_id,
5         SUM(sales) AS total_sales
6     FROM orders
7     GROUP BY customer_id
8 )
9
10 SELECT *
11 FROM customer_sales
12 WHERE total_sales >
13 (
14     SELECT AVG(total_sales)
15     FROM customer_sales
16 );
```

	customer_id	total_sales
1	AA-10315	5563.56
2	AA-10645	5086.935
3	AB-10060	7755.62
4	AB-10105	14473.571
5	AC-10450	5527.846
6	AD-10180	6106.88
7	AG-10675	3489.0396
8	AG-10900	4510.797
9	AH-10075	3250.337

UTF-8